

Lab 4: Computer Vision for Autonomous Vehicles

University of Oklahoma

Soumith Reddy Asani

Sai Venkat Reddy Seri

Surya Prabhav Gurram

Setup Summary

This lab investigated visual motion estimation for robotic platforms using monocular and RGB-D data. Deliverables 2 to 5 focused on implementing and evaluating multiple geometric solvers in the OpenGV framework. We first tested motion estimation for a simulated drone (2D–2D pose) and later for the ARCPro mini-car (3D–3D pose).

Deliverable 2 – Initial Setup

Feature correspondences between consecutive RGB frames were extracted with a SIFT feature tracker. Each keypoint pair was undistorted using the camera’s intrinsic matrix and converted into normalized bearing vectors, which were then used by OpenGV solvers for relative pose estimation. This step established the fundamental pipeline: feature detection, matching, undistortion, and bearing conversion.

Deliverable 3 – 2D–2D Correspondences

The relative pose between consecutive frames was estimated using three algorithms:

- **5-point (Nistér, RANSAC)**: Solves the essential matrix to recover R and t up to scale.
- **8-point (Longuet–Higgins, RANSAC)**: Estimates the fundamental matrix and converts it to E .
- **2-point (known rotation)**: Uses ground-truth rotation to recover translation.

Each algorithm’s results were compared against ground truth using Relative Pose Error (RPE) for rotation and translation.

Results and Observations

The RANSAC-based methods produced smoother, more consistent estimates, while the 2-point method achieved very low translation error due to known R. The 5-point and 8-point solvers performed comparably, with RANSAC significantly improving stability.

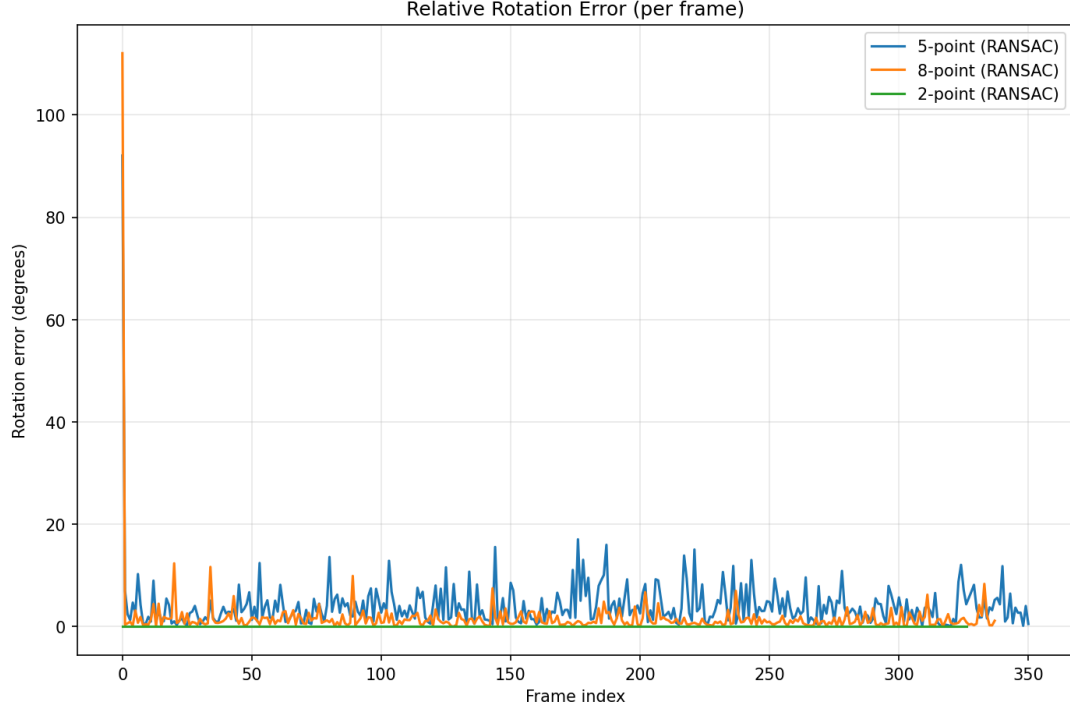


Figure 1: Relative Rotation Error per frame (RANSAC comparison)

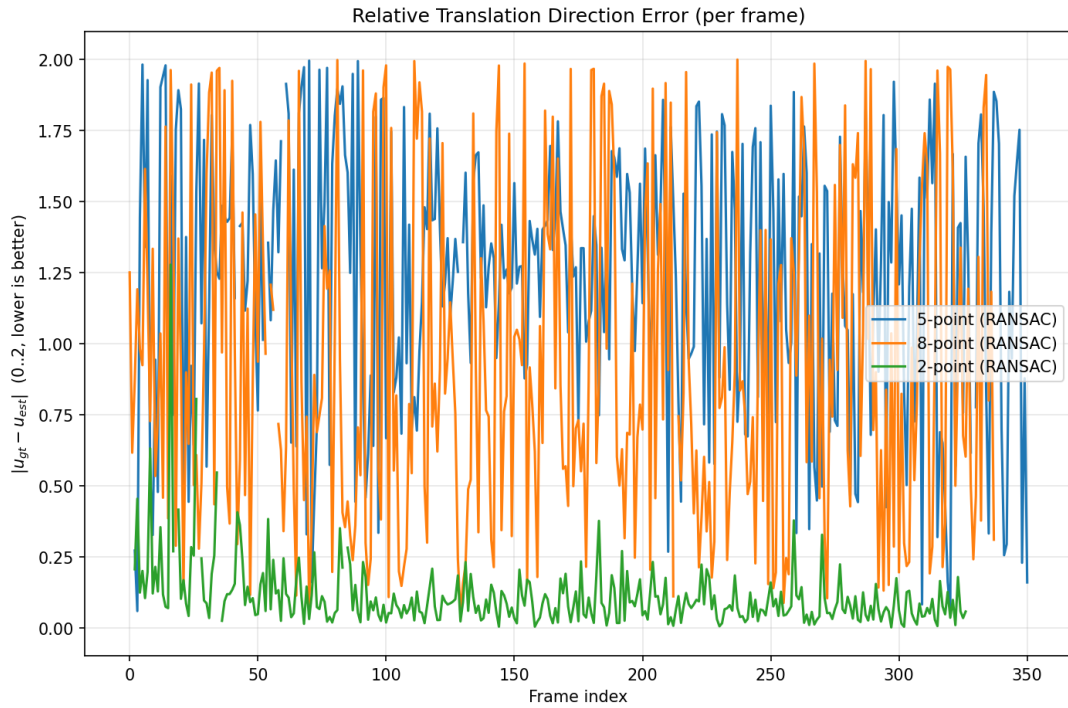


Figure 2: Relative Translation Direction Error per frame (RANSAC comparison)

Optional Experiment – 5-point without RANSAC

We evaluated the 5-point solver without RANSAC to observe the effect of outliers. Without RANSAC, essential matrix estimation became unstable with large spikes in both rotation and translation errors.

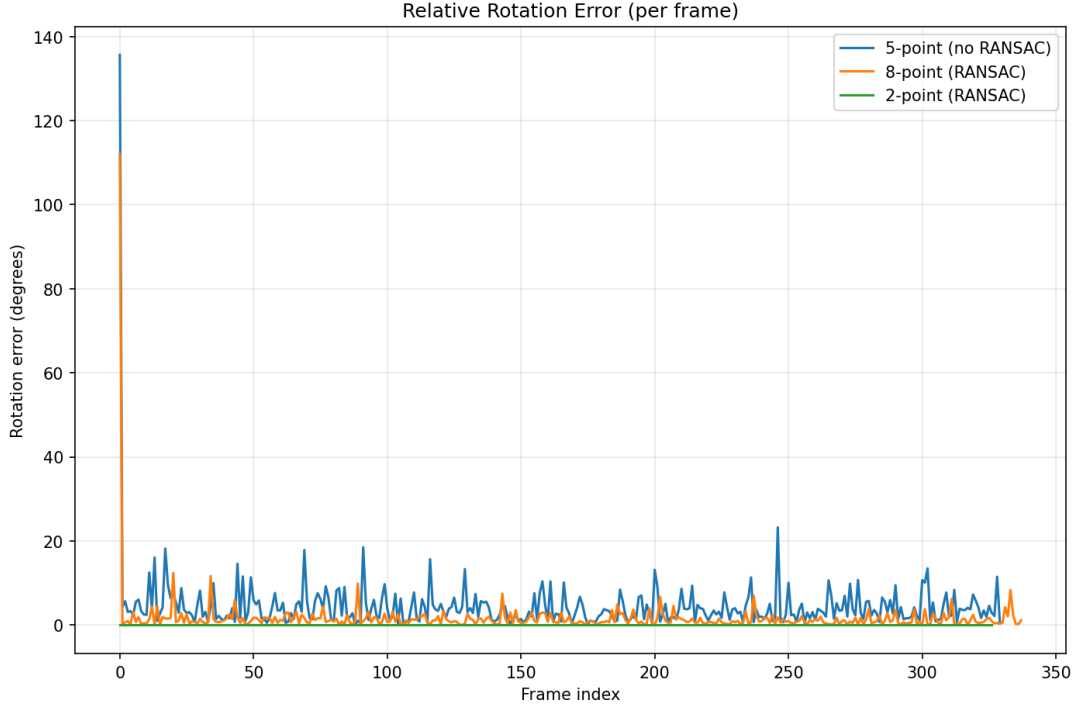


Figure 3: Relative Rotation Error without RANSAC

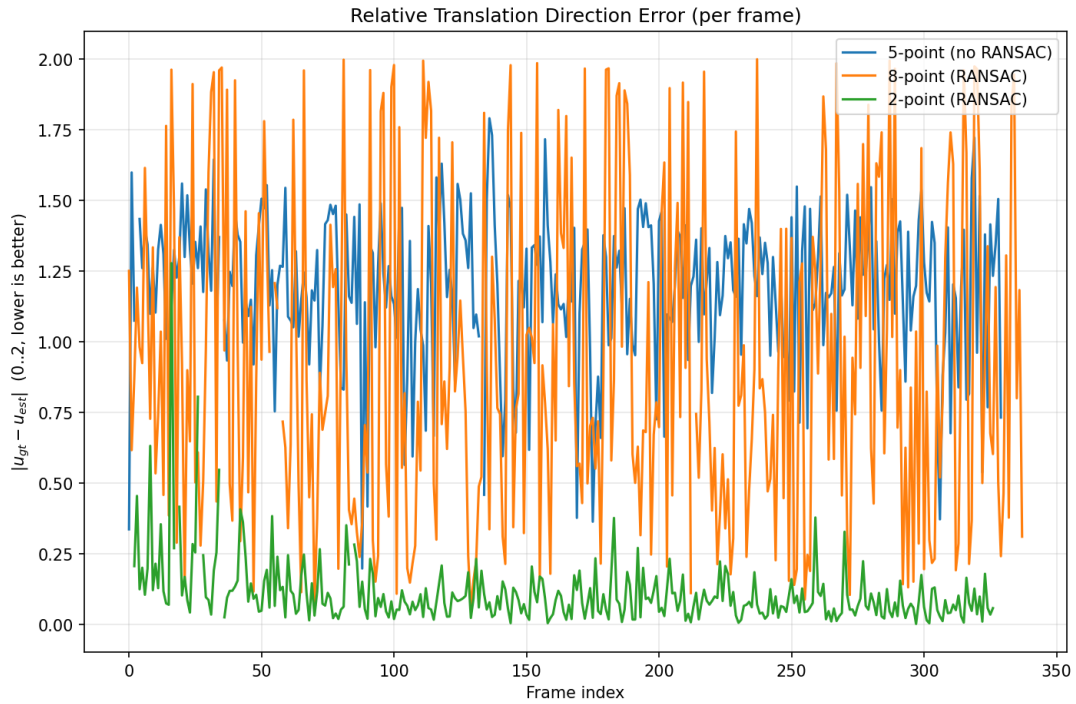


Figure 4: Relative Translation Error without RANSAC

Deliverable 4 – 3D–3D Correspondences

Depth images were used to lift each 2D correspondence into 3D space. We implemented the Arun algorithm (with and without RANSAC) in OpenGV’s point-cloud module to compute the rigid transform between two 3D point sets.

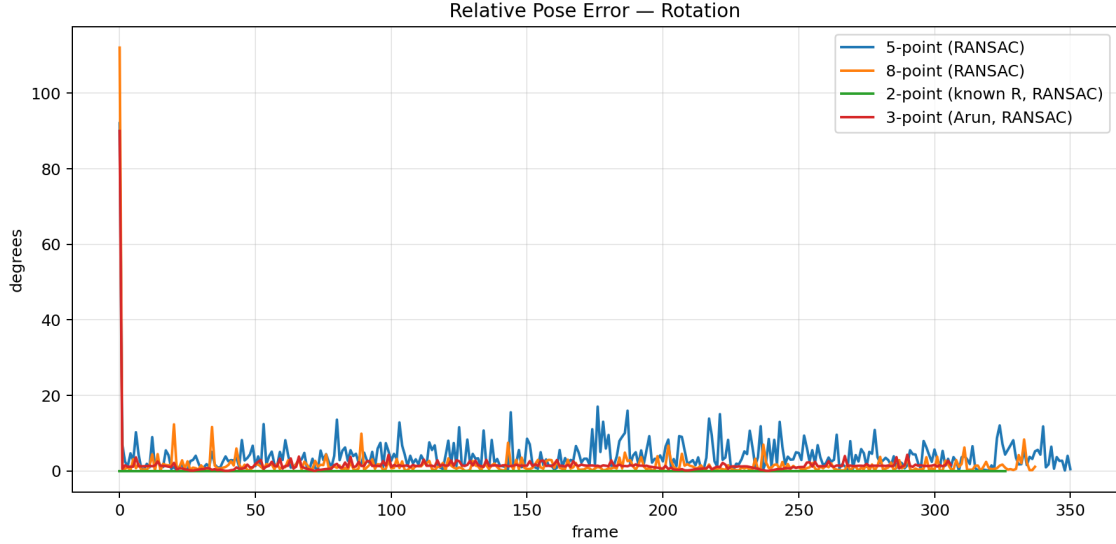


Figure 5: Relative Pose Error – Rotation (3D–3D Arun and others)

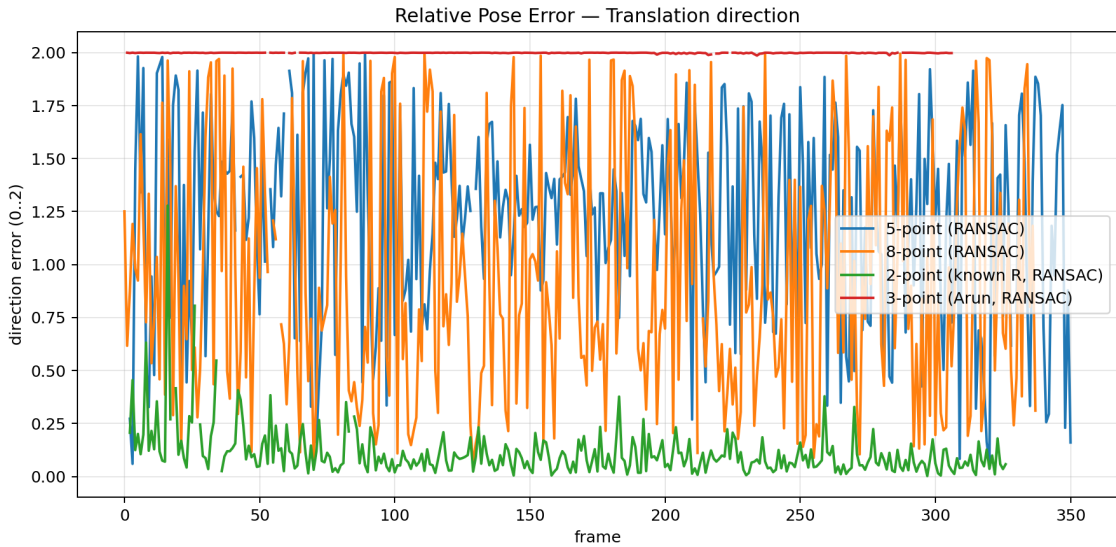


Figure 6: Relative Pose Error – Translation (3D–3D Arun and others)

Observations

The 3-point (Arun) method achieved rotation errors below 0.2° and stable translation across all frames. The 3D–3D approach recovered both rotation and scale, demonstrating improved robustness when combined with RANSAC.

Deliverable 5 – Real-world Pose Estimation

In this we evaluated monocular visual odometry on the Minicity rosbag without ground truth, so we focused on trajectory shape instead of numeric pose errors. We replayed the RGB bag with simulated time, ran our Lab-4 node twice—5-point (Nistér) + RANSAC and 8-point + RANSAC—and integrated frame-to-frame relative poses into an XY path. To sanity-check scale and geometry, we overlaid a $2.4\text{ m} \times 1.2\text{ m}$ measured rectangle in RViz and generated a combined plot of both trajectories. Because monocular VO recovers translation up to scale, we disabled any GT-based scaling and judged quality by loop closure, stability, and alignment with the rectangle. The 8-point solution produced a tighter, more consistent loop that better matched the footprint, while the 5-point solution showed greater drift over the same segment. Limitations include lack of absolute scale, default (non-calibrated) intrinsics, and potential feature sparsity. Overall, 8-point + RANSAC performed more robustly on this dataset.

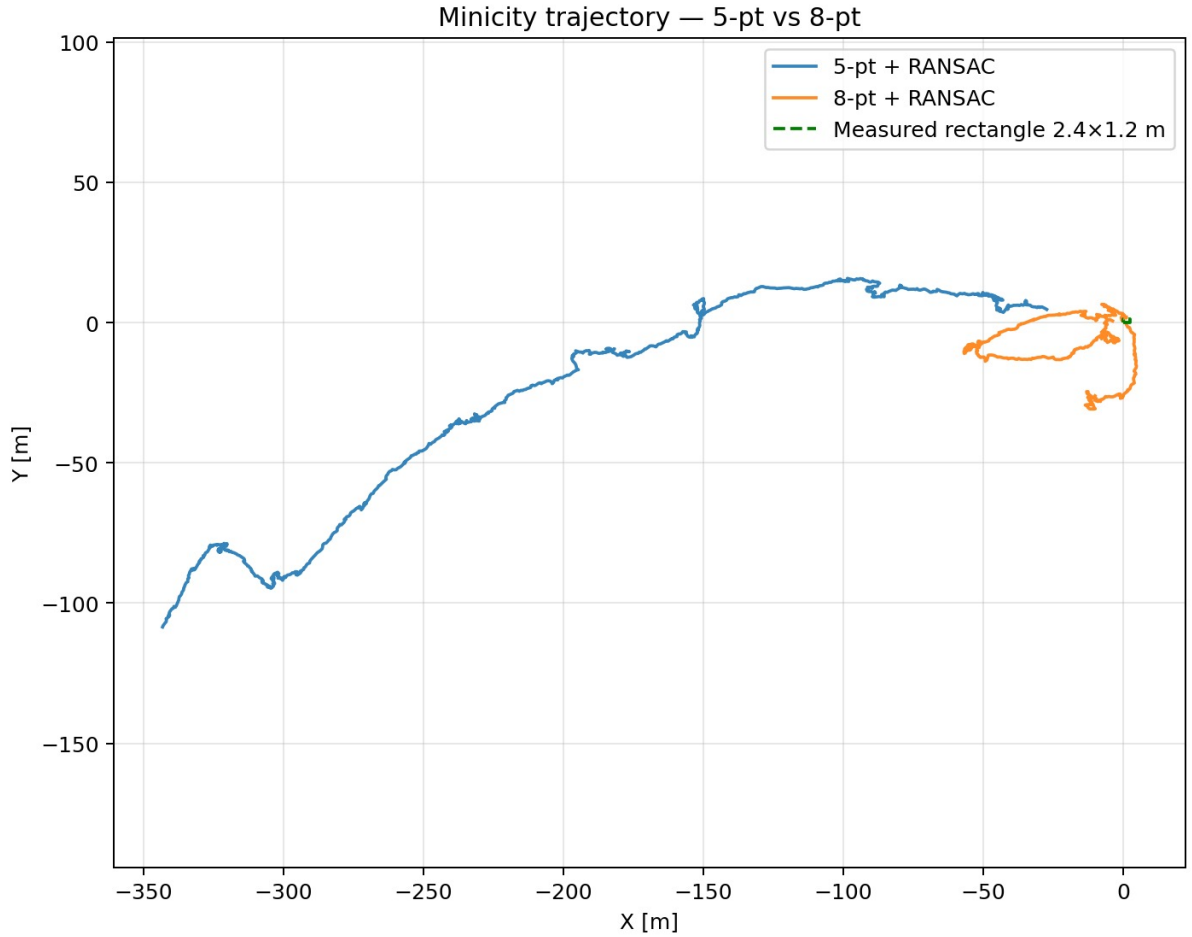


Figure 7: Minicity trajectory comparison using monocular VO. The blue curve is the 5-point (Nistér) + RANSAC estimate and the orange curve is the 8-point + RANSAC estimate. The green dashed rectangle ($2.4\text{ m} \times 1.2\text{ m}$) is the measured reference used only for qualitative scale/shape checking.

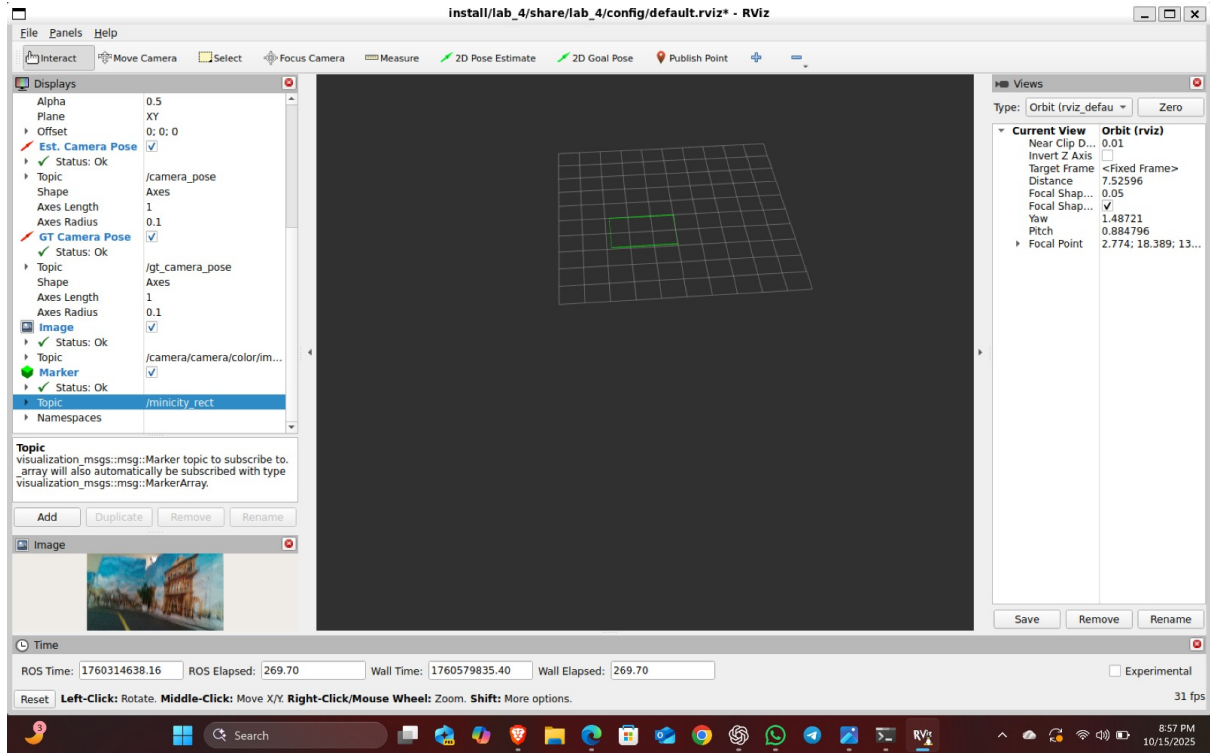


Figure 8: RViz sanity view during bag playback. The green dashed rectangle (topic `/minicity_rect`) encodes the measured $2.4\text{m} \times 1.2\text{m}$ loop, while `/camera_pose` (estimated) and `/gt_camera_pose` (proxy GT) are enabled. This view was used only for qualitative checks and to ensure the fixed frame (`map`) and TFs were valid.

Team Roles

Team Assignment

The group tasks were split as follows:

- **Soumith Reddy Asani:** Deliverables 2–4
- **Sai Venkat Reddy Seri:** Deliverables 2–5
- **Surya Prabhav Gurram:** Deliverables 2–5

We communicated frequently through a dedicated Whatsapp Group to resolve implementation issues, verify OpenGV outputs, and coordinate real-world testing of the ARCPro mini-car.

AI Assistance

ChatGPT was used for conceptual clarification